

Two Techniques:

The Potential Method

Divide and Conquer

Savage Chickens

by Doug Savage



www.savagechickens.com



"WE ALREADY HAVE QUITE A FEW PEOPLE WHO KNOW HOW TO DIVIDE. SO ESSENTIALLY, WE'RE NOW LOOKING FOR PEOPLE WHO KNOW HOW TO CONQUER."

For this section only

- I maintain a webpage with various links.

Exclusive to us, ***bookmark this now:***

<https://mahdi.ch/eecs376>

- Canvas: Only for video recordings
- ***Anonymous*** feedback (goes to me only!):

<https://mahdi.ch/eecs376/feedback>

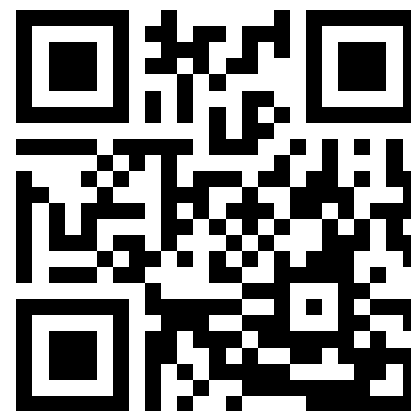
- ***Livestream*** link
(not guaranteed to always work):

<https://mahdi.ch/eecs376/zoom>

- Add a slide about yourself here
(only if you attend this section):

<https://mahdi.ch/eecs376/meet-greet>

- Find your name card



Last time:

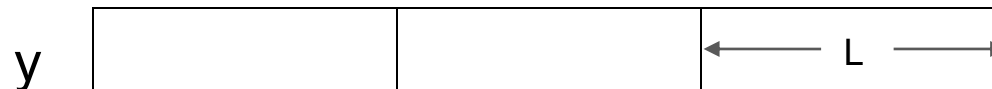
The Tiling problem (aka gcd)

Input: n-digit integers $x \geq y \geq 0$, but not both = 0.

$$\begin{aligned} n &= \log_{10} y \\ y &= 10^n \end{aligned}$$

Output: largest integer L that divides both x and y (aka greatest common divisor)

In other words: largest integer tile size that can exactly tile a path of length x and a path of length y

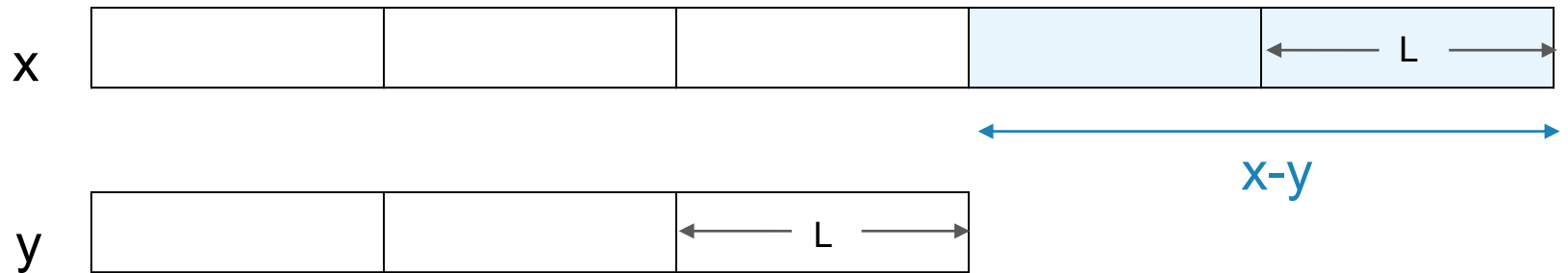


I want to tile
two paths but
only buy one
size of tile



Euclid's idea! (actually it was known before Euclid)

Claim: $\gcd(x, y) = \gcd(x-y, y)$.



Proof. We'll show that for any L : L divides x and $y \Leftrightarrow L$ divides $x-y$ and y .

1. ($\Leftarrow$) Suppose L divides both $x-y$ and y .

Why does this mean L divides x ?

2. ($\Rightarrow$) Suppose L divides both x and y .

Why does this mean L divides $x-y$?

Euclid's (more general) idea!

$$\gcd(x, y) = \gcd(x-y, y)$$

$$= \gcd(x-2y, y)$$

$$= \gcd(x-3y, y)$$

$$= \gcd(x-ky, y) \text{ for any } k \text{ such that } x-ky \geq 0$$

$$= \gcd(x \bmod y, y)$$

Now we have a smaller instance of the same problem.

What should we do?

Euclid's *recursive* gcd algorithm

- $\text{Euclid}(x, y)$ // we'll assume that $x > y \geq 0$.

Base case:

If($y=0$) then return(x)

General case:

Return($\text{Euclid}(y, x \bmod y)$)

Analyzing Algorithms

In general, analyzing an algorithm requires two components:

- Proving correctness. (Prove by induction, proof omitted)
- Proving a running time bound.

Euclid's algorithm: Running Time

We will show that, in each recursive call to Euclid, the x , y arguments are *collectively* decreasing *very quickly*

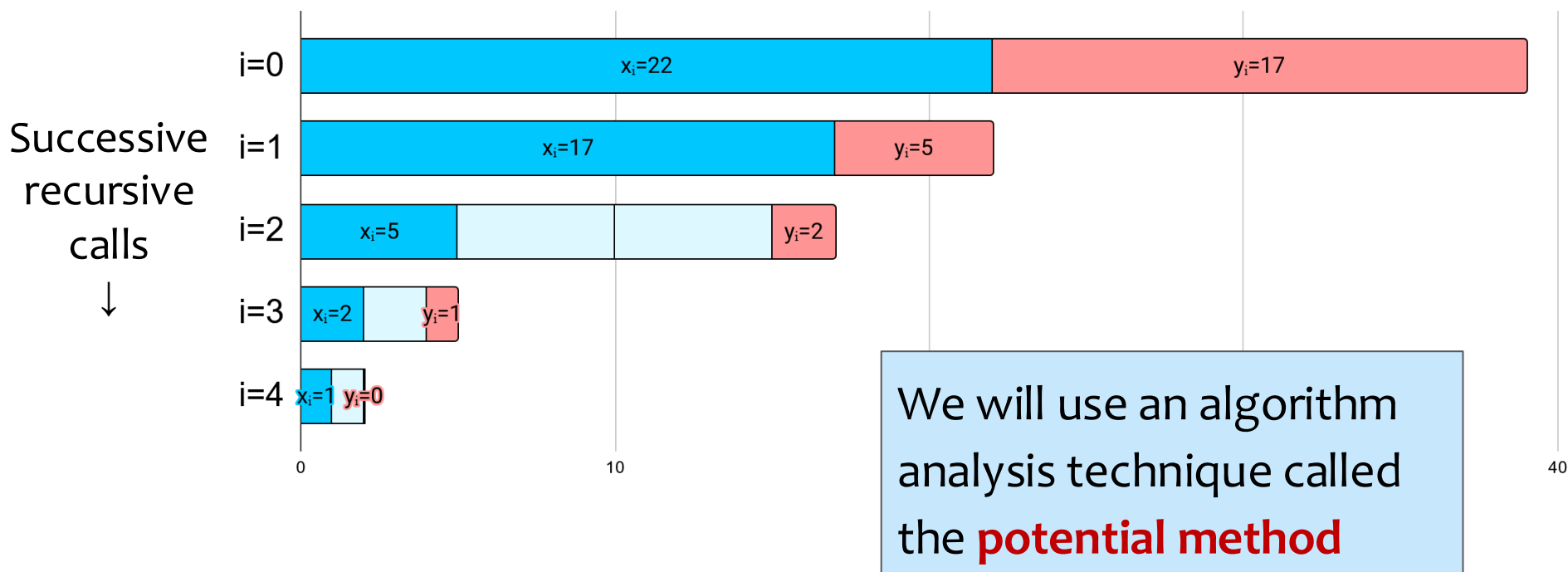


Successive
recursive
calls



Euclid's algorithm: Running Time

We will show that, in each recursive call to Euclid, the x , y arguments are *collectively* decreasing *very quickly*



The Potential Method

Today we will analyze the running time of Euclid's algorithm using the **potential method**.

... But first, a toy example to illustrate this method

The potential method appeared in a movie!

<https://www.youtube.com/watch?v=oCoKdZXldyU>

A Flipping Game

- 3 x 3 board covered with two-sided chips:  / 
- Two players, **R (row)** and **C (column)**, alternately perform “flips”:
 - **R** flips every chip in some **row** with $\# \text{ Ohio State } > \# \text{ M }$
 - **C** flips every chip in some **column** with $\# \text{ Ohio State } > \# \text{ M }$
- If no flip is possible, then the game ends.
- **Question:** Must the game always end?



R flips
row 3



C flips
column 1



Let's formalize this reasoning into a general-purpose method

Intuitively, a **potential function argument** says:
If I start with some amount of water in a leaky bucket,
then soon water must stop leaking out.



Ingredients of the argument:

1. Define the “unit of time” e.g. one iteration of an algorithm
2. Define a measure of the “amount of water in the bucket.”
This is the **potential function S_i** $\leftarrow$ amount of water in bucket at time i
3. Provide an upper bound on how big **S_0** can be.
4. Provide a lower bound on how small **S_i** values can be (often 0)
5. Prove that the bucket “leaks quickly”. I.e. that **S_i** decreases by at least some amount per unit time.
6. Use this to upper bound the total number of units of time.

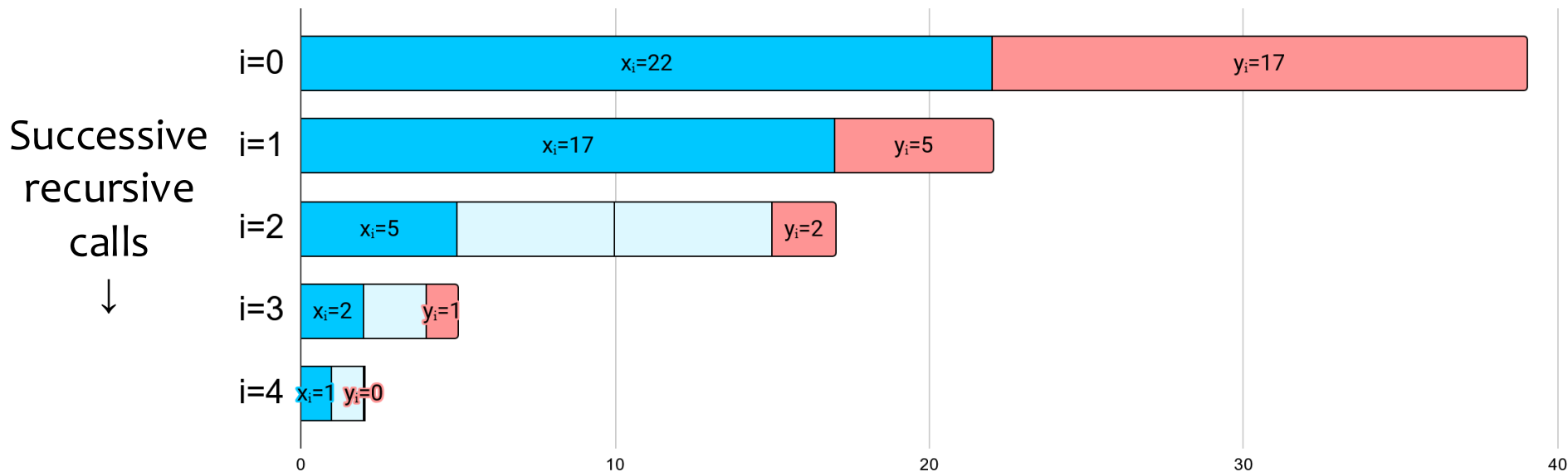
Analyzing the Flipping Game via a Potential Function

1. Unit of time = one player's turn.
2. Define the **potential function** $S_i = \#$  chips at turn i .
3. Note that $S_0 \leq 9$
4. S_i can never be negative. $S_i \geq 0$
5. At every turn the value of S_i decreases by at least **1**, i.e.: $S_{i+1} \leq S_i - 1$
6. This implies that the total number of turns is at most $S_0 \leq 9$.

Now let's apply the potential method to Euclid's Algorithm...

Euclid's algorithm: Running Time

We will show that, in each recursive call to Euclid, the x , y arguments are *collectively* decreasing *very quickly*



Reminders about Euclid(x, y):

- $x > y \geq 0$
- $\text{Return}(\text{Euclid}(y, x \bmod y))$

Analyzing Euclid's Algorithm via Potential Function y_i

1. Unit of time = one recursive call.
2. Define the **potential function** $S_i = y_i$.
3. $S_0 = y$
4. $S_i \geq 0$
5. At every recursive call the value of S_i decreases by at least **1**. Why?
6. Thus, the total number of calls to Euclid is at most $S_0 = y$.

Reminders about Euclid(x,y):

- $x > y \geq 0$
- `Return(Euclid(y, x mod y))`

But we already knew this! Recall that the brute-force algorithm from last lecture already achieved y iterations.

This is a ba-a-a-a-d
analysis of a good
algorithm

Conclusion: We need a function S_i that decreases by more.



Analyzing Euclid's Algorithm via Potential Function x_i

1. Unit of time = one recursive call.
2. Define the **potential function** $S_i = x_i$.
3. $S_0 = x$
4. $S_i \geq 1$
5. At every recursive call the value of S_i decreases by at least **1**. Why?
6. Thus, the total number of calls to Euclid is at most $S_0 = x-1$.

Reminders about Euclid(x,y):

- $x > y \geq 0$
- `Return(Euclid(y, x mod y))`

But we already knew this! Recall that the brute-force algorithm from last lecture already achieved y iterations (which is fewer than x).

This is a ba-a-a-a-d
analysis of a good
algorithm

Conclusion: We need a function S_i that decreases by more.



Let's convince ourselves that potential functions $S_i = y_i$ and $S_i = x_i$ give $\exp(n)$ running time bounds
(for time unit = 1 recursive call)

$S_i = y_i$: What is an example of x, y values (≥ 10) such that S_i decreases by only 1, i.e. $y - y_1 = 1$?

$S_i = x_i$: What is an example of x, y values (≥ 10) such that S_i decreases by only 1, i.e. $x - x_1 = 1$?

Analyzing Euclid's Algorithm via a Potential Function

Finding the right potential function can be a fine art.

It turns out that even though neither $S_i = y_i$ nor $S_i = x_i$ work, $S_i = x_i + y_i$ does!

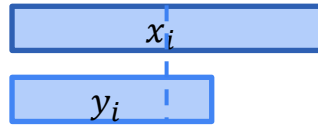
1. Unit of time = one recursive call.
2. Define the **potential function** $S_i = x_i + y_i$.
3. $S_0 = x + y$
4. $S_i \geq 1$ for all i
5. But how does S_{i+1} compare to S_i ?

Analyzing Euclid's Algorithm via a Potential Function

Potential function: $S_i = x_i + y_i$

Goal: $S_{i+1} \leq c S_i$

Case 1: $y_i > \frac{1}{2} x_i$



Reminders about Euclid(x,y):

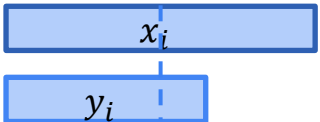
- $x > y \geq 0$
- $\text{Return}(\text{Euclid}(y, x \bmod y))$

Analyzing Euclid's Algorithm via a Potential Function

Potential function: $S_i = x_i + y_i$

Goal: $S_{i+1} \leq c S_i$

Case 1: $y_i > \frac{1}{2} x_i$



$$\begin{aligned} S_{i+1} &= x_{i+1} + y_{i+1} \\ &= y_i + (x_i - y_i) \\ &= x_i \\ &= c \left(\frac{3}{2} \right) x_i \\ &= c \left(x_i + \frac{1}{2} x_i \right) \\ &< c (x_i + y_i) \\ &= c S_i \end{aligned}$$

Reminders about Euclid(x,y):

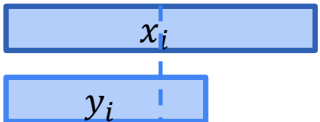
- $x > y \geq 0$
- $\text{Return}(\text{Euclid}(y, x \bmod y))$

Analyzing Euclid's Algorithm via a Potential Function

Potential function: $S_i = x_i + y_i$

Goal: $S_{i+1} \leq (2/3) S_i$

Case 1: $y_i > \frac{1}{2} x_i$



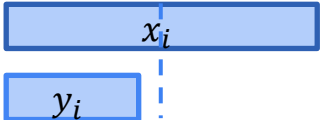
$$\begin{aligned}
 S_{i+1} &= x_{i+1} + y_{i+1} \\
 &= y_i + (x_i - y_i) \\
 &= x_i \\
 &= \left(\frac{2}{3}\right) \left(\frac{3}{2}\right) x_i \\
 &= \left(\frac{2}{3}\right) \left(x_i + \frac{1}{2} x_i\right) \\
 &< \left(\frac{2}{3}\right) (x_i + y_i) \\
 &= \left(\frac{2}{3}\right) S_i
 \end{aligned}$$

Either way, $S_{i+1} < \left(\frac{2}{3}\right) S_i$

Reminders about Euclid(x,y):

- $x > y \geq 0$
- $\text{Return}(\text{Euclid}(y, x \bmod y))$

Case 2: $y_i \leq \frac{1}{2} x_i$



$$\begin{aligned}
 S_{i+1} &= x_{i+1} + y_{i+1} \\
 &= y_i + (x_i \bmod y_i) \\
 &< y_i + y_i \\
 &= 2y_i \\
 &= \left(\frac{2}{3}\right) (3y_i) \\
 &= \left(\frac{2}{3}\right) (2y_i + y_i) \\
 &\leq \left(\frac{2}{3}\right) (x_i + y_i) \\
 &= \left(\frac{2}{3}\right) S_i
 \end{aligned}$$

Analyzing Euclid's Algorithm via a Potential Function

Finding the right potential function can be a fine art.

It turns out that even though neither $S_i = y_i$ nor $S_i = x_i$ work, $S_i = x_i + y_i$ does!

1. Unit of time = one recursive call.
2. Define the **potential function** $S_i = x_i + y_i$.
3. $S_0 = x + y$
4. $S_i \geq 1$ for all i
5. At every recursive call the value of S decreases by at least a **multiplicative factor**, specifically $S_{i+1} < (2/3) \cdot S_i$ for all i .
6. **Claim:** Step 5 implies: total # recursive calls is $O(n)$. (need to prove)

Consequence of Step 6:

poly(n) by grade-school division

Final running time = $O(n) \cdot (\text{time to compute } x \bmod y)$

Analyzing Euclid's Algorithm via a Potential Function

Claim: If $S_{i+1} \leq (2/3) \cdot S_i$ for all i , then total # recursive calls is $O(n)$.

Proof. Observe $S_i \geq 1$.

In particular, our final step $S_k \geq 1$.

$$\text{So, } 1 \leq S_k \leq (2/3)^k \cdot (x+y)$$

$$\Leftrightarrow (3/2)^k \leq (x+y)$$

$$\Leftrightarrow k \leq \log_{3/2}(x+y)$$

$$< \log_{3/2}(2 \cdot 10^n)$$

$$= \log_{3/2}(2) + n \cdot \log_{3/2}(10)$$

$$= O(n)$$

$$\begin{aligned} S_0 &= x+y, \\ S_1 &\leq (2/3) \cdot (x+y), \\ S_2 &\leq (2/3)^2 \cdot (x+y), \\ &\dots \\ S_i &\leq (2/3)^i \cdot (x+y) \end{aligned}$$

Recap: Analyzing Euclid's Algorithm via a Potential Function

Finding the right potential function can be a fine art.

It turns out that even though neither $S_i = y_i$ nor $S_i = x_i$ work, **$S_i = x_i + y_i$** does!

1. Unit of time = one recursive call.
2. Define the **potential function $S_i = x_i + y_i$** .
3. **$S_0 = x + y$** .
4. **$S_i \geq 0$** for all i .
5. At every recursive call the value of **S** decreases by at least a **multiplicative factor**, specifically **$S_{i+1} \leq (2/3) \cdot S_i$** for all i . (proved!)
6. **Claim:** Step 5 implies: total # recursive calls is $O(n)$. (proved!)

Consequence of Step 6:

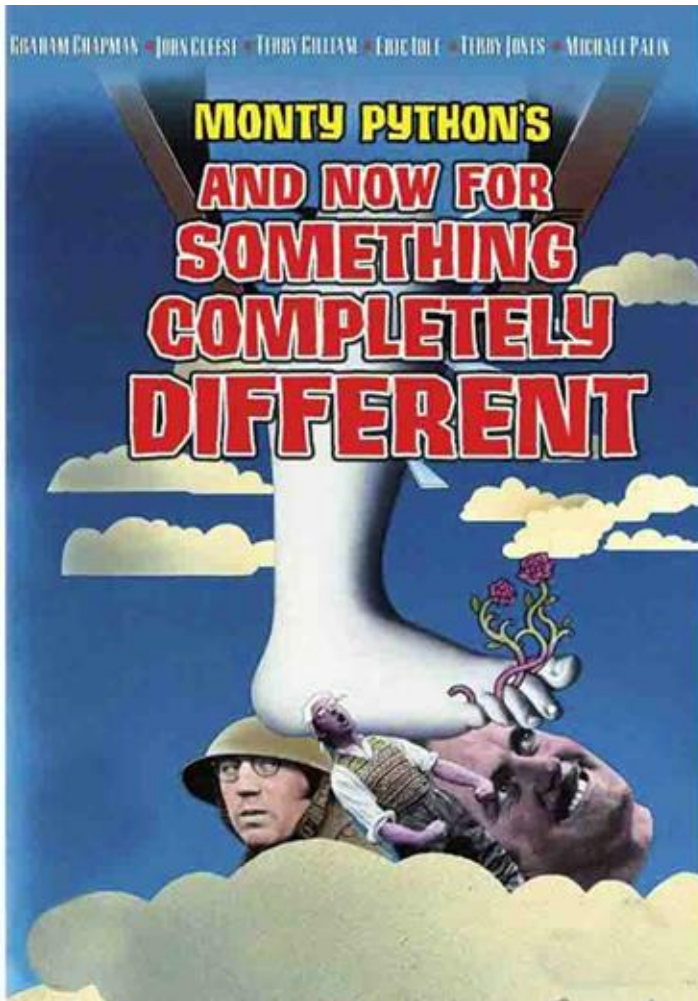
poly(n) by grade-school algorithm

Final running time = $O(n) \cdot$ (time to compute $x \bmod y$)

When to use the potential method

Part of the challenge (and fun) of algorithm design is figuring out when to use which technique.

General intuition: The potential method could be useful when some quantity seems to be increasing or decreasing over the execution of the algorithm, getting you closer and closer to termination.



A Design
Technique:
Divide and
Conquer

Overview: Divide-and-Conquer Algorithms

Main Idea:

1. **Divide** the input into smaller sub-problems
2. Solve each sub-problem *recursively*
3. **Conquer:** Combine the sub-solutions to get a solution for the original input

Designing the Algorithm + Proving Correctness: an “art”

- Depends on problem structure, ad-hoc, creative

Running time Analysis: “mechanical”

- Express runtime using a recurrence
- Can often solve using the “Master Theorem”

Mergesort

Input:
Array of
numbers

1	2	3	4					n
6	0	4	6	...	...	...	...	3

Output:
Sorted

1	2	3	4					
0	3	4	6	6	...	...	...	...

Discovered by John von Neumann in 1945



Mergesort



Unsorted array of length n

Mergesort

Unsorted array of length n



Mergesort

Unsorted array of length n

Unsorted array of length $n/2$

Unsorted array of length $n/2$

Mergesort

Unsorted array of length n

Unsorted array of length $n/2$

Unsorted array of length $n/2$



Mergesort

Unsorted array of length n

Unsorted array of length $n/2$

Unsorted array of length $n/2$

Sorted array of length $n/2$

Sorted array of length $n/2$

Mergesort

Unsorted array of length n

Unsorted array of length $n/2$

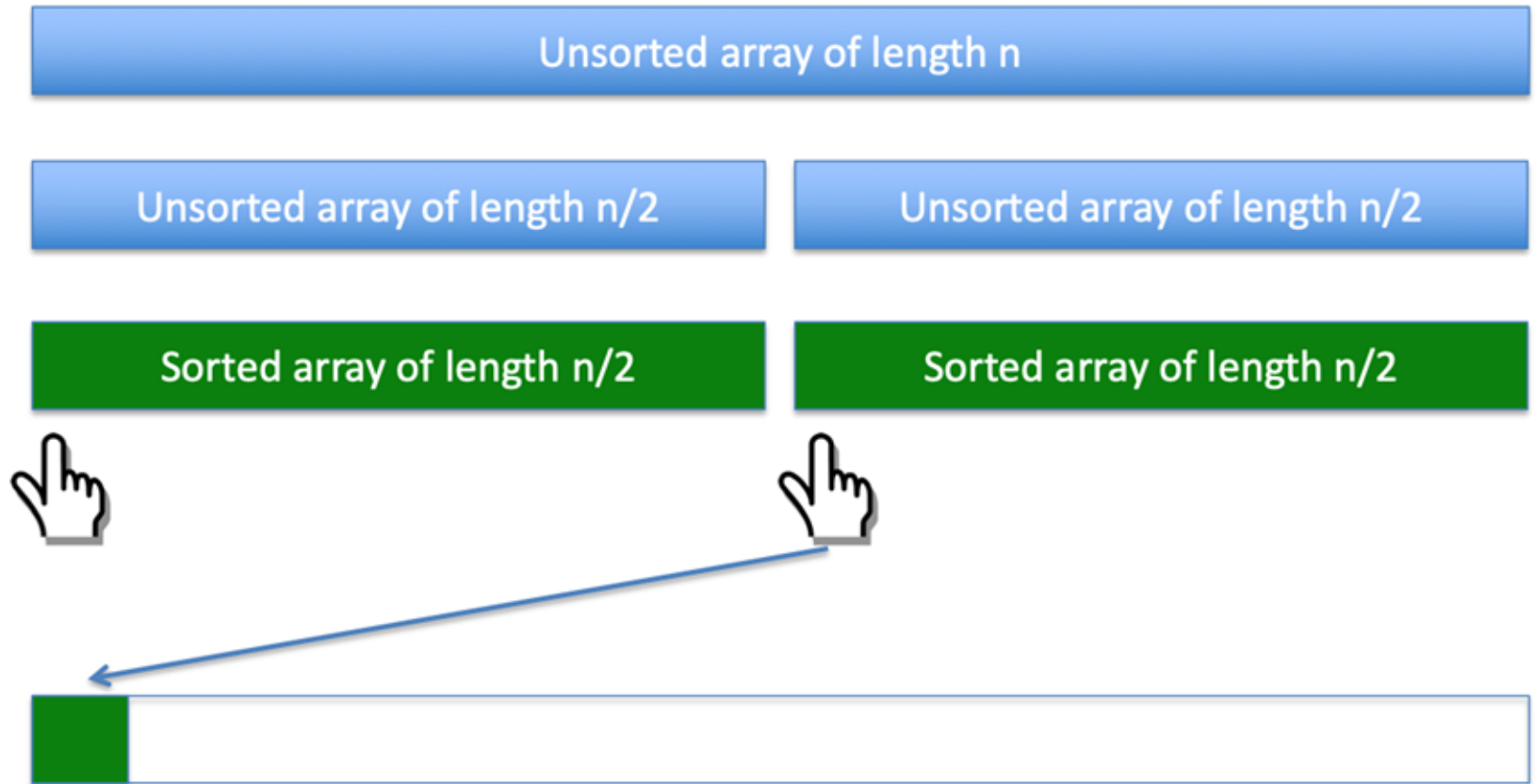
Unsorted array of length $n/2$

Sorted array of length $n/2$

Sorted array of length $n/2$



Mergesort



Mergesort

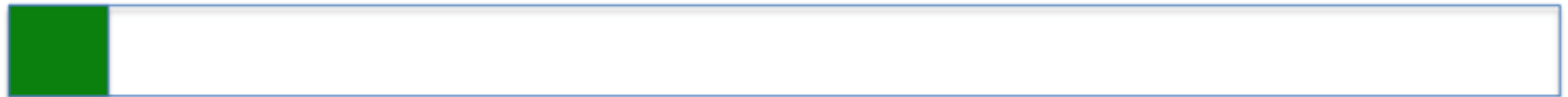
Unsorted array of length n

Unsorted array of length $n/2$

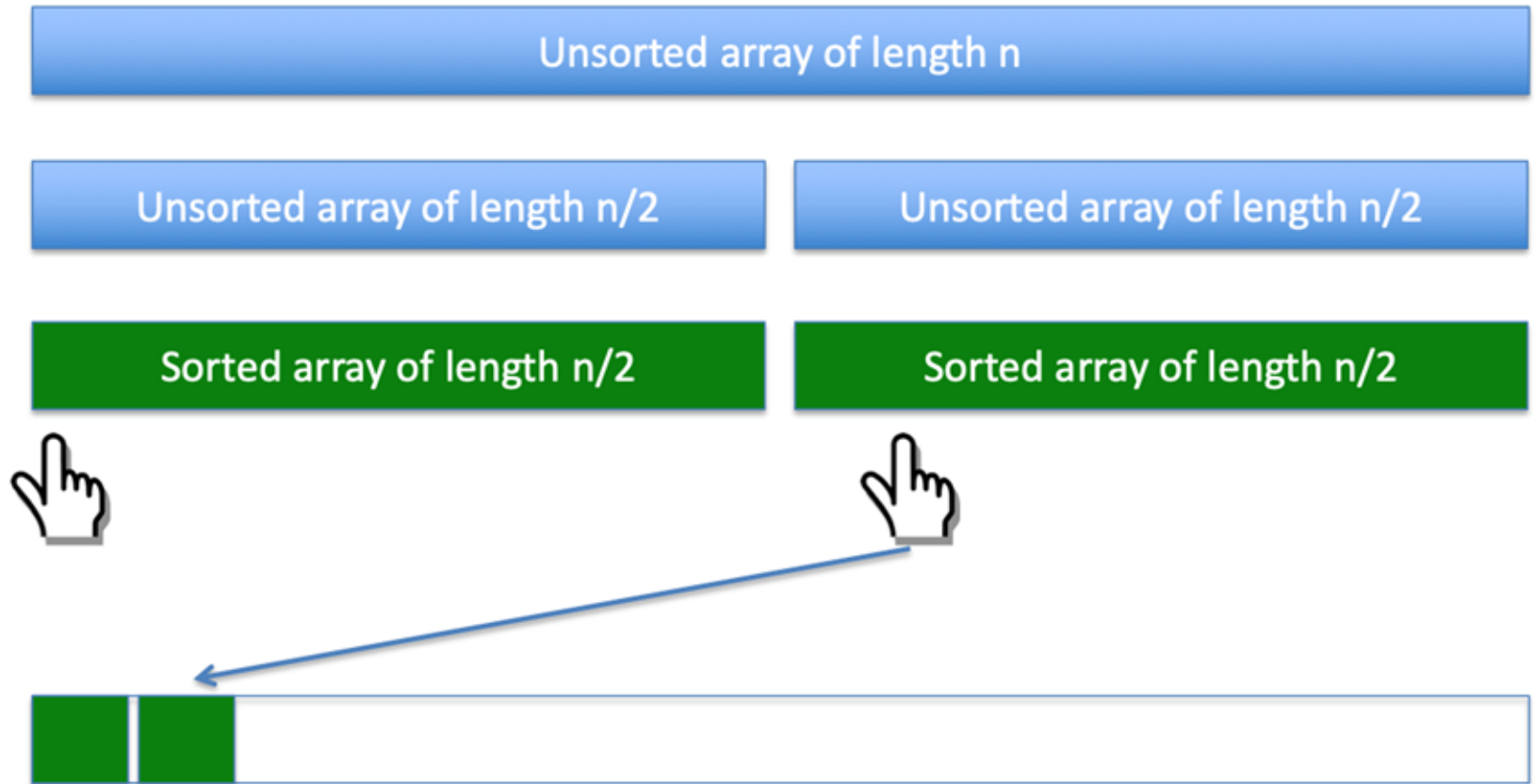
Unsorted array of length $n/2$

Sorted array of length $n/2$

Sorted array of length $n/2$



Mergesort



Mergesort

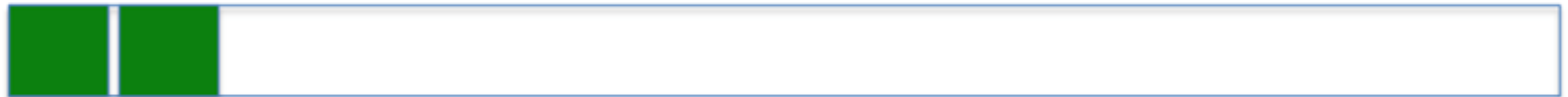
Unsorted array of length n

Unsorted array of length $n/2$

Unsorted array of length $n/2$

Sorted array of length $n/2$

Sorted array of length $n/2$



Mergesort

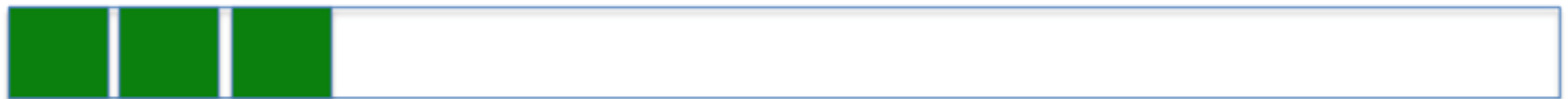
Unsorted array of length n

Unsorted array of length $n/2$

Unsorted array of length $n/2$

Sorted array of length $n/2$

Sorted array of length $n/2$



Mergesort

Unsorted array of length n

Unsorted array of length $n/2$

Unsorted array of length $n/2$

Sorted array of length $n/2$

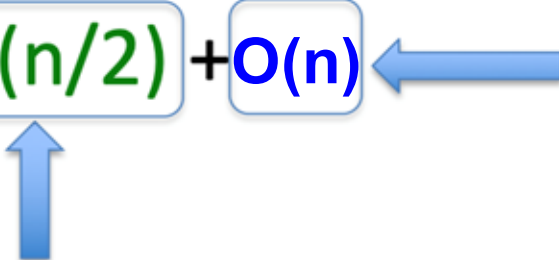
Sorted array of length $n/2$



How long does it take
to merge two sorted
arrays, each of length
 $n/2$?

Recurrences and Running Times

$T(n)$ = worst case running time of mergesort on input of length n

$$T(n) = 2 T(n/2) + O(n)$$


Two recursive calls on problems of size $n/2$

Merge two arrays of size $n/2$